Danmarks
Tekniske
Universitet

Software Enginering Project Report 1

Authors

Mathias Kjølbye - s240803
Nicolai Søgaard - s234146
Nat Skovlund - s245203
Vedanta Sinha - s245010
Viktor Munk - s240335

11. april 2026

Indhold

1 Introduction (Vedanta) 1

2 Glossary (Nat) 2

3 Use Case Diagram (Viktor) 4
3.1 Actors 4
3.2 Main Use Cases 4
3.3 Alternative And Error Secenarios 5

4 Detailed Use Cases and Sequence Diagram (Vedanta) 6
4.1 Use Case 1: Creating Projects (Vedanta) 6
4.2 Use Case 2: Creating an Activity (Vedanta) 6
4.3 Use Case 3: Add Developer to Activity (Nat) 7
4.4 Use Case 4: Register Time on Activity (Nat) 8
4.5 Use Case 5: Generate Report(Nicolai) 9
4.6 Use Case 6: Register Absence (Nicolai) 11
4.7 Use Case 7: View Available Developers(Mathias) 12
4.8 Use Case 8: Assign Project Leader(Mathias) 12
4.9 Use Case 9: Set project Deadline(Viktor) 13
4.10 Use Case 10: View Project Progress(Viktor) 14

5 UML Diagram (Nicolai) 16
5.1 UML domains (Nicolai) 16
5.1.1 Project 16
5.1.2 Activity 17
5.1.3 Developer 17
5.1.4 Time Registration 17
5.1.5 Absence 17

6 Program Design (Vedanta) 18

7 Discussion (Vedanta) 18

8 Appendix 19

List of Figures 19

1 Introduction (Vedanta)

The purpose of the project is to create a software that allows an organization to manage its projects and employees. This requires a software that is capable of multiple nuanced use cases that are vital for maintaining a solid overview of an organization. These use cases consist of allocating employees to projects and activities, while simultaneously logging the absence of employees or the completion of projects and activities.

This software primarily allows admins in an organization to allocate project managers while have a quick overview of the project structure and progress. This software also offers simple yet effective planning tools to enable project managers. Similarly, it lets developers to register for absence or to generate reports that they can provide to the project managers.

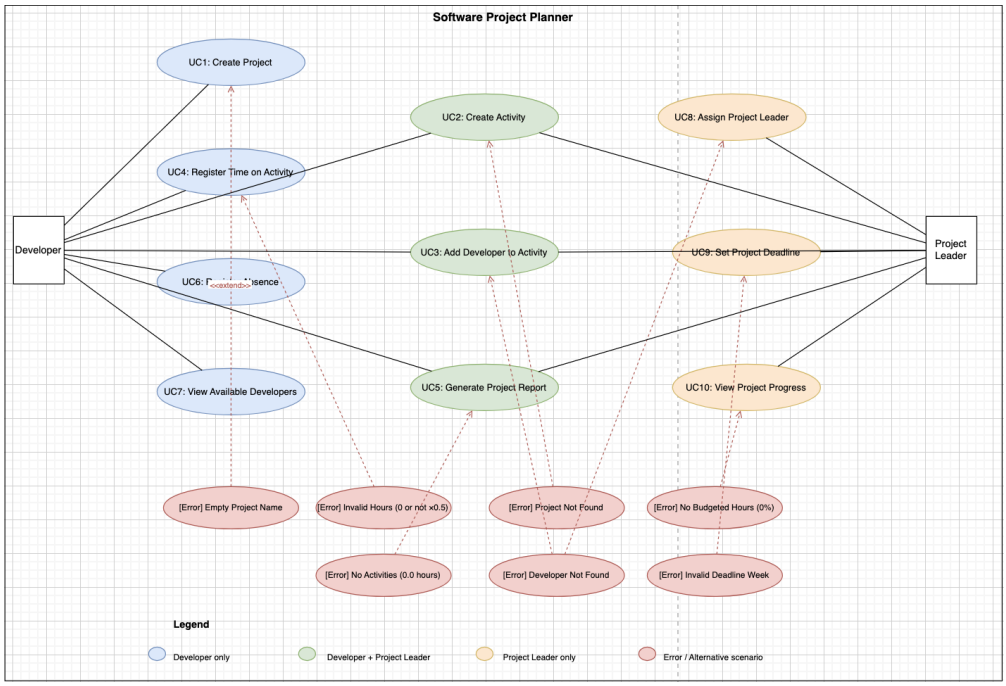
2 Glossary (Nat)

Term	Definition
Absence	Times when a developer is not available to be assigned work
Activity	Something that has to be completed on a certain project.
Actor	The users of the system that have various ways to interact with it
Admin	Someone with special privileges to edit settings and view more information.
Application	See definition of "System"
Available Developer	A developer who can be assigned to a given Activity.
Budgeted and Budgeted hours	The time that an Activity is allowed to take.
Console UI	The User interface
Deadline	When a specific project should be finished
Developer Activity	Someone who is working on the project. Something that has to be completed on a certain project.
Disk Based Storage	Things such as HDD's and SSD's. they are generally slower than in memory storage.
Encoded Logic	The Behavior of the system in various scenarios.
Error	The message send to the user, and behavior of the system when something unintentional happens.
Extend relationship	A use case that is not a part of standard operations, but describes the behavior when something unintended happens, for example when trying to access a project that does not exist.
Flushed	When the data stored is not only set as overwriteble, but is deleted.
Forwards	Sends the information on to another part of the program.
Implementation	The act of making the project usable for its stated purpose.

In memory storage	Fast to access memory, that is flushed when depowered.
Initialized	When variables in a program are first assigned values.
Key-Operations	The uses of the system described in the use cases.
Layer	Word used to describe categories to differentiate between different parts of the program, in relation to what kind of function they serve.
Main Project Entity	The main program of the system.
MainWindow	The name of the program for the UI.
Product	Something that is created, usually for the purpose of being sold.
Project	The development of a software product
Project Leader	The project leader has the final say in regarding how the project should run, and how many developers should be active
Project Manager	Synonyms with "Project Leader"
Project Status	How close the project is to completion, and how much time has been spent.
Scenario	Something that might happen
Sequence Diagram	A model of what is being sent between different parts of the program, in what order.
System	Our product that lets the user manage their projects
Task	See definition of "Activity"
Time Registration	The act of logging the time used on a task or project.
UML-Diagram	A diagram describing how the different parts of the system, called classes, intact and what they contain.
Use Case	A specification of how the project should act in certain scenarios, and what it generally has to be able to do.
Work Hours	The time used on a task or project.

3 Use Case Diagram (Viktor)

Figure 1 Presents a system-level use case diagram for the software Project planner. The diagram provides an overview of the key operations supported by the system and the actors that interact with them.



Figur 1: User-Case diagram over scenarios/fail scenarios

3.1 Actors

The system has two actors:

- **Developer** Is a regular employee who can create projects, register time on activities, register absence, view available developers, create activities, add developers to activities, and generate project reports.
- **Project Leader** Is a developer who has been assigned as responsible for a project. In addition to the developer's operations, the project leader can assign project leaders, set project deadlines, and view project progress. The Project Leader can also be a developer.

3.2 Main Use Cases

The ten main use cases are grouped into three categories based on who can perform them, as indicated by color in the diagram:

- **Developer only** (Blue): Create Project, Register Time on Activity, Register Absence, View Available Developers
- **Project Leader only** (orange): Assign Project Leader, Set Project Deadline, View Project Progress.
- **Developer and Project Leader**(green): Create Activity, Add Developer to Activity, Generate Project Report.

3.3 Alternative And Error Secenarios

The Diagram also includes alternative/error scenarios (red), which extend the main use cases using the "Extend"relationship. These represent situations where the system must reject invalid input:

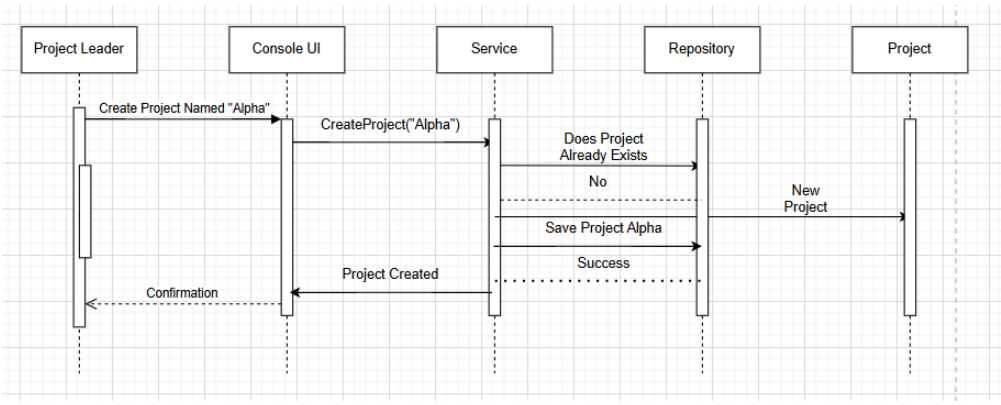
- **Empty Project Name** Extends Create Project when the name is blank.
- **Invalid Hours** Extends Register Time on Activity when hours are zero or not a multiple of 0.5.
- **Project Not Found** Extends Create Activity when the given project does not exit.
- **Developer Not Found** Extends Add Developer to Activity and Assign Project Leader when the given developer does not exit.
- **Invalid Deadline Week** Extends Set Project Deadline when the week number is outside the valid range of 1 - 53
- **No Activities** Extends User-case 5 (Generate Report) when the project has no activities, resulting in a report showing zero hours.
- **No Budget Hours** Extends user-case 10 (View Project Progress) when no hours have been budgeted for the project, resulting in a progress of 0

4 Detailed Use Cases and Sequence Diagram (Vedanta)

It is important to keep in mind the possible use cases that are required from such a software in a company. The following subsections will, in detail, explain the behaviour expected from this system.
Along with a description, there will also be scenarios that will help later in order to test the application during implementation.

4.1 Use Case 1: Creating Projects (Vedanta)

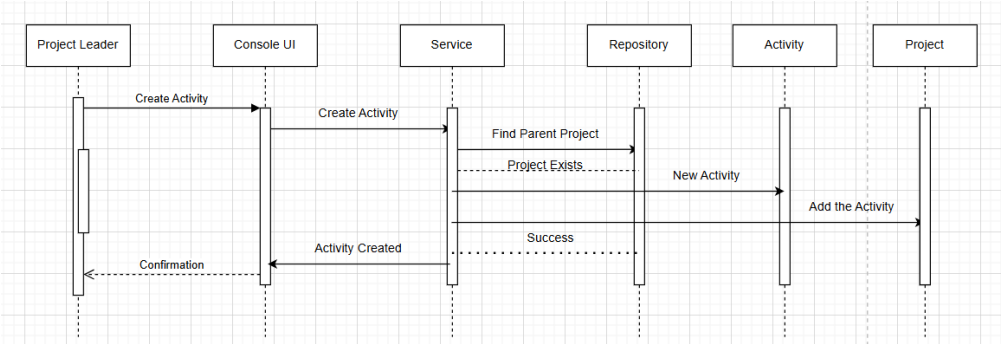
This use case is to let admins create projects on the systems. This is the first use case, as most of the other use cases, such as registering a developer, requires a project to already be present on the system.
The first scenario for this use case is an admin creating a project named 'Alpha', and there are no other projects named Alpha, the system should create a project named Alpha and store it. However, if an admin tries to create a project and forgets to give it a name, it should give an error. Furthermore, if an admin tries to create a project with a name that already exists, it should also give an error.



Figur 2: Sequence diagram for Main use case 1

4.2 Use Case 2: Creating an Activity (Vedanta)

Projects are large and often require finishing several different activities. This use case allows project managers to create activities inside a project that developers can finish. Finishing activities would naturally lead the project to completion.
The first scenario for this use case is simply a project manager creating an activity that is required for a project's completion. On the other hand, if a project manager attempts to create an activity without first specifying the project, the system should return an error.



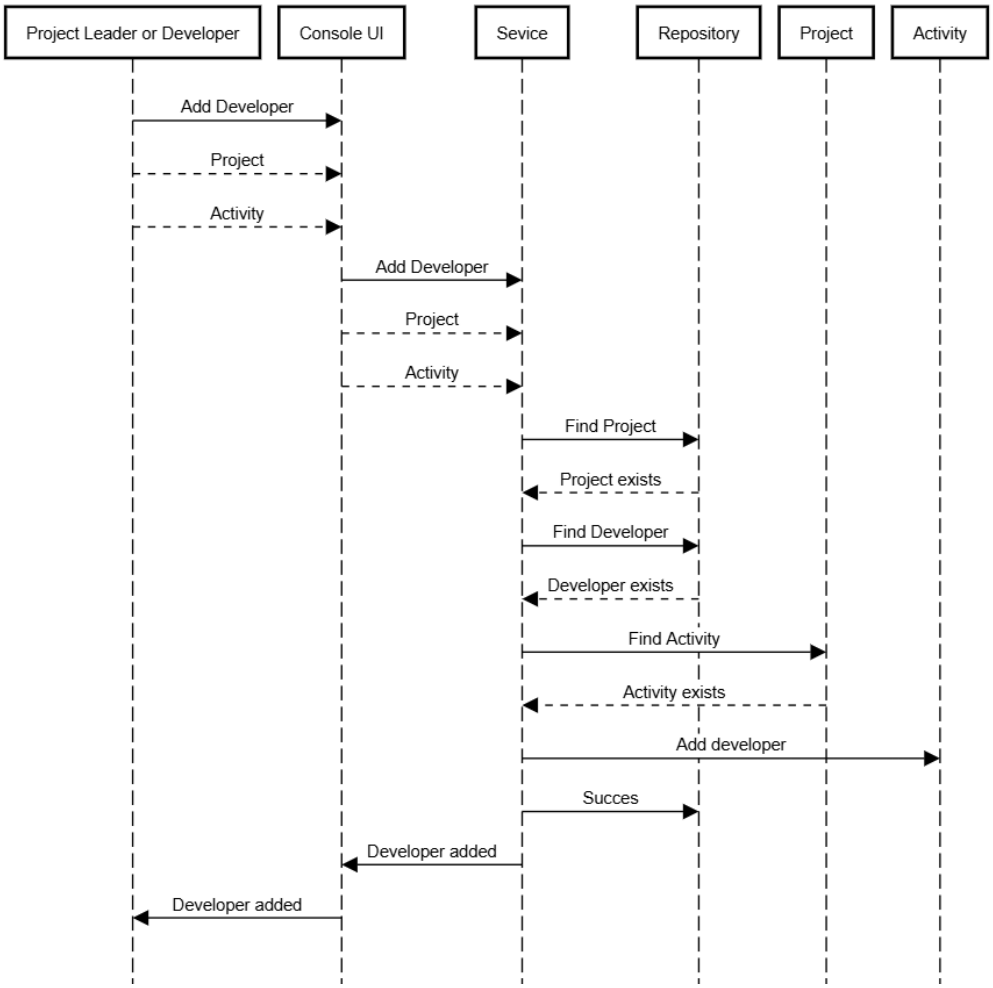
Figur 3: Sequence diagram for Main use case 2

4.3 Use Case 3: Add Developer to Activity (Nat)

This use case allows a developer or project leader to assign a developer to a specific activity within a project. This enables the system to track which developers are responsible for which activities, and is used when checking developer availability for a given week.

Main scenario: The developer and the activity both exist in the system. the developer is successfully assigned to the activity.

Alternative scenario: The specified developer does not exist in the system. The system returns an error "Developer not found".



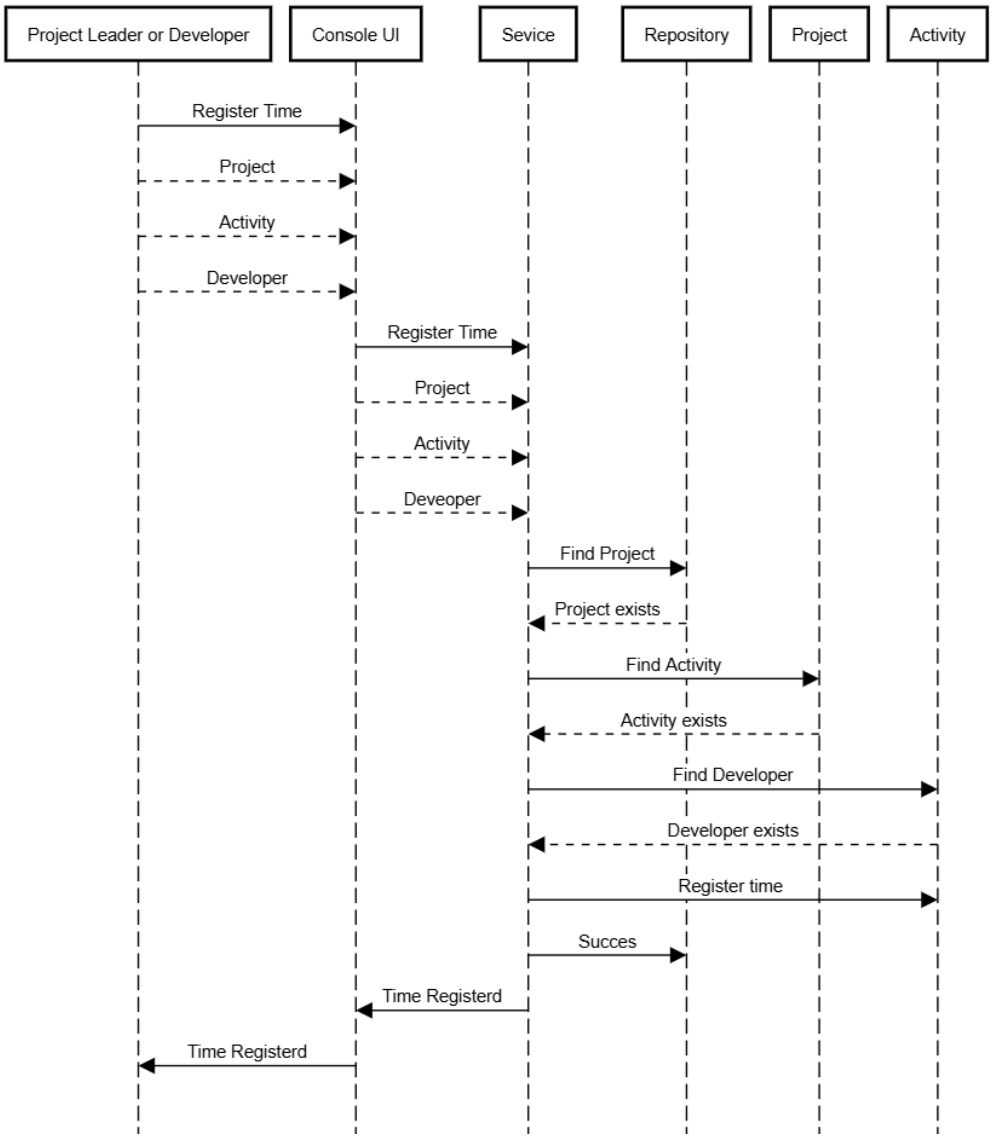
Figur 4: Sequence diagram for Main use case 3

4.4 Use Case 4: Register Time on Activity (Nat)

It is important for project managers to be aware of the amount of time that the developers are using in order to finish the work assigned to them. This is crucial for them to gauge the effectiveness of the developers.

Main scenario: for this use case is simply a developer using 5 hours to finish a certain activity within a project, and then registering for those 5 hours.

Alternative scenario: If a developer tries to register hours for an activity they are not assigned to, the system should give an error.



Figur 5: Sequence diagram for Main use case 4

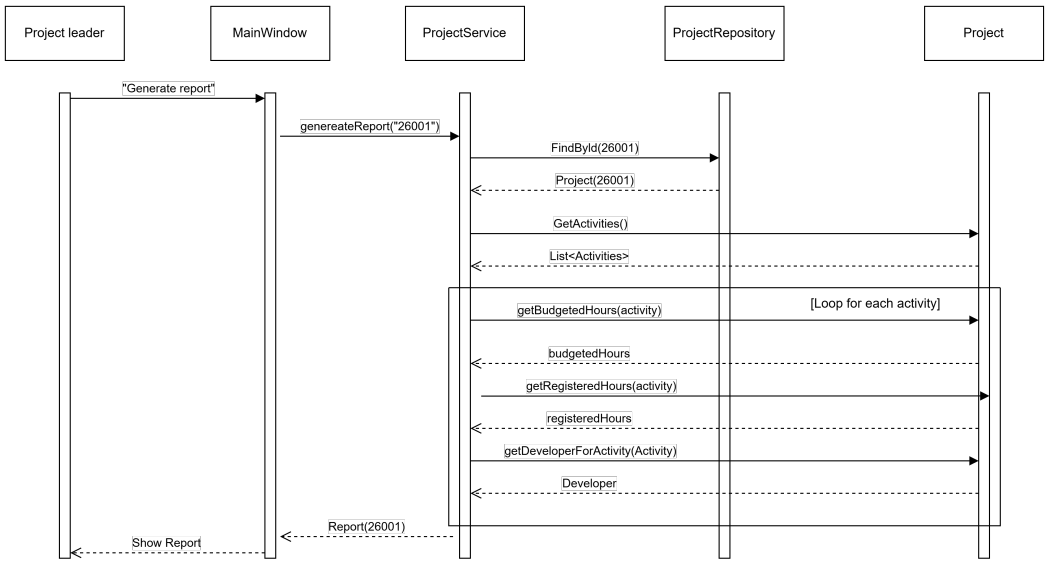
4.5 Use Case 5: Generate Report(Nicolai)

For a given project, an activity inside the project can be added. This activity is required to have a expected or budgeted amount of hours to be done. When a developer then registers x -amount of hours on this activity, it is logged. This is done for every activity inside all projects made in the program. When a project leader generates a report on a project, all registered and logged hours will then be shown.

First Scenario
An activity "UI"in a project is budgeted 100 hours.
A developer with intials "NS"has logged 30 hours into the activity.

When the project leader wants a report on the project, a report
is generated and shown in `MainWindow`.
The report shows that the developer "NS"has registered 30 hours into activity "UI"

Second scenario
A project has no logged hours in their activities
When a report on the project is created. No amount of hours has gone into the project.
The report is blank



Figur 6: Sequence diagram for use case 5

In 6, it is illustrated how a report is created.

The project leader uses th UI (`MainWindow`) to `"Generate Report"`. `MainWindow` calls for the `generateReport()` through the `ProjectService`. From here the project is retrieved from the `ProjectReposity`. The project you want to find is returned, followed by a request for the activity-list for the given project.

For each activity, the service retrieves the budgeted hours and the registered time entries. It loops this for every activity inside a project.

"E.g. An activity has a budget of 100 hours, and the developer has registered 30 hours on that activity."

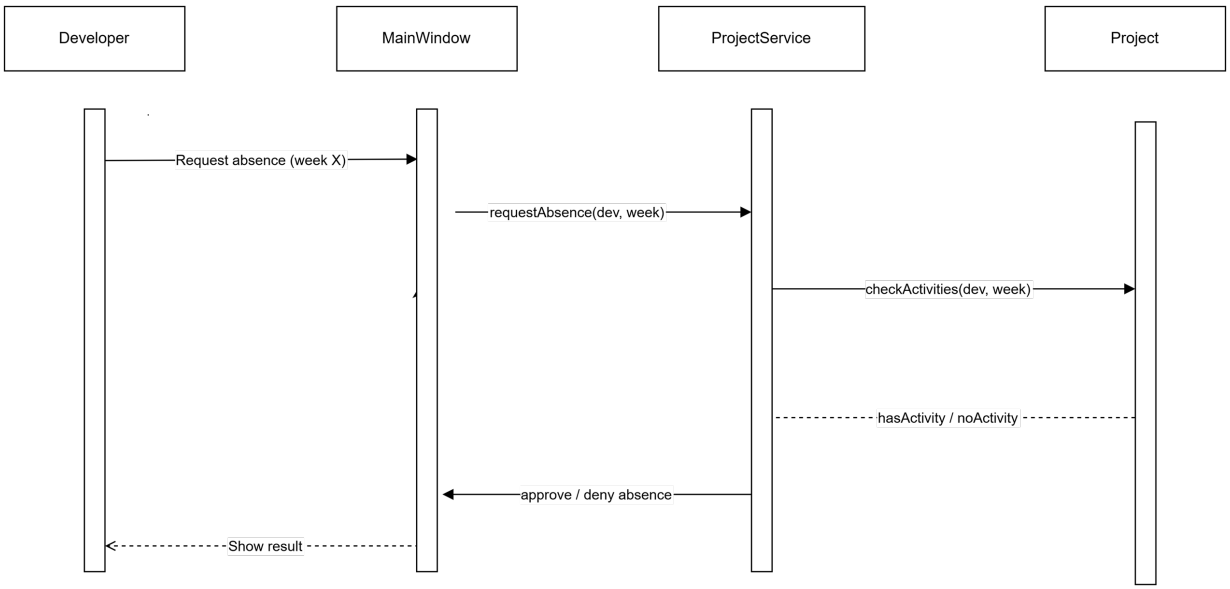
Based on this information `ProjectService` creates a report, which is then shown in the UI.
In case of the failed scenario 2, a report will be generated, but no hours will be logged and will return blank.

4.6 Use Case 6: Register Absence (Nicolai)

In case of absence, the system has to register the absence of the developer. This includes type of absence and expected time until the developer is available again:

First Scenario
A Developer registers vacation in a given week.
The developer is not assigned to any activity in that week.
The developer request for vacation is granted, and the developer is marked as *unavailable* during that week.

Second Scenario:
A Developer registers vacation in a given week.
The developer is assigned to an activity in that week.
The Vacation request is denied.



Figur 7: Sequence diagram for Use Case 6

Figure 7 shows the sequence of interactions when a developer requests absence.
The process starts when a developer `"requests absence"` in the `MainWindow`. `requestAbsence()` is called through `ProjectService`, passing on the requested week and developer who wants it.

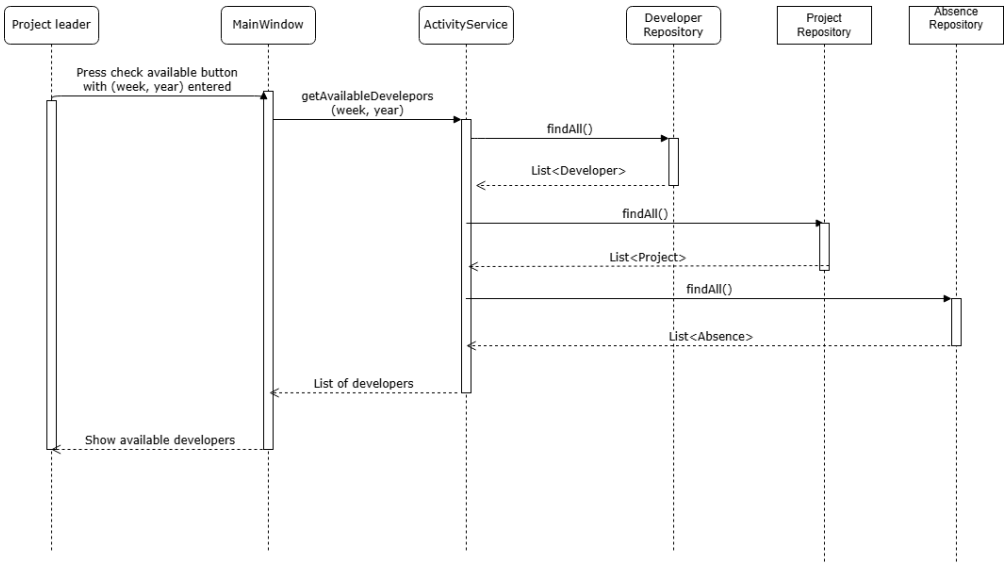
`ProjectService` checks wheter the developer is assigned to any activittis in the given week by calling `checkActivities()` in the project object. The project return whether the develeoper has any activities or not.

Based on the information found through this call, The `ProjectService` evaulates the request. If the developer is not assigned to any activities, the absence is approved. Otherwise, the request is denied. The result will be shown to the developer in the `MainWindow` UI.

This use case describes features that are intended future functionality. Not all things mentioned are part of the current implementation.

4.7 Use Case 7: View Available Developers(Mathias)

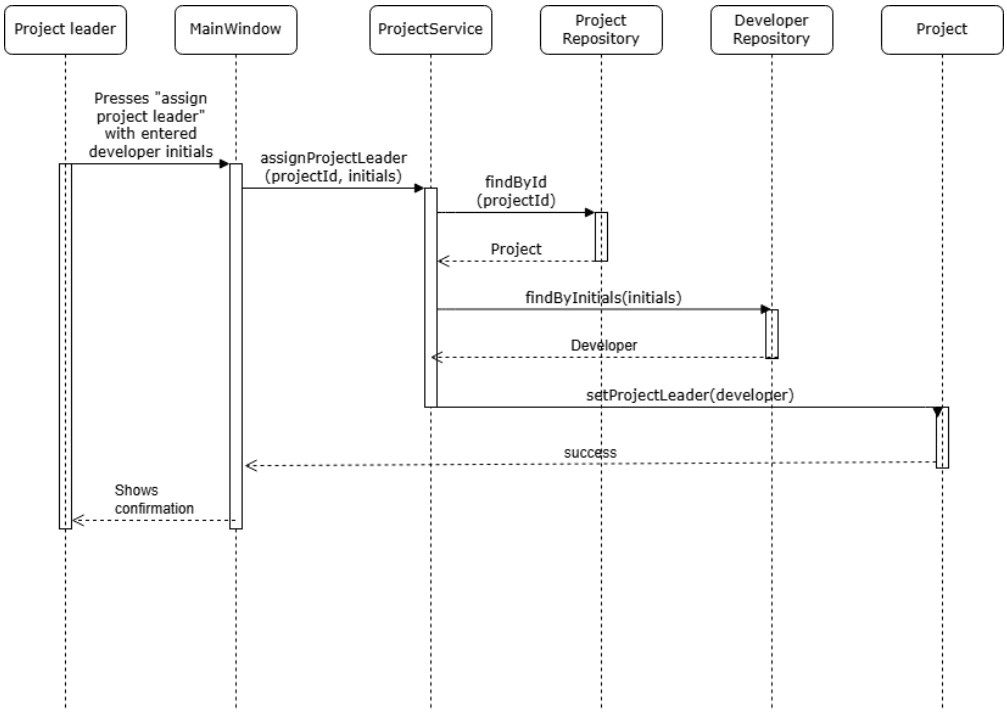
This use case describes the functionality for viewing available developers. It is tested through two scenarios that demonstrate situations in which a developer should not be included in the list of available developers. The first scenario covers a developer who is already assigned to an activity in the relevant time period, the second covers a developer who has registered vacation during that period.



Figur 8: Sequence diagram for use case 7

4.8 Use Case 8: Assign Project Leader(Mathias)

This use case describes the functionality for assigning a project leader. It is tested through two scenarios. The first scenario verifies that an existing developer can be assigned as the project leader of an existing project. The second scenario verifies that the assignment is rejected if the specified developer does not exist, in which case the system raises an error instead.



Figur 9: Sequence diagram for use case 8

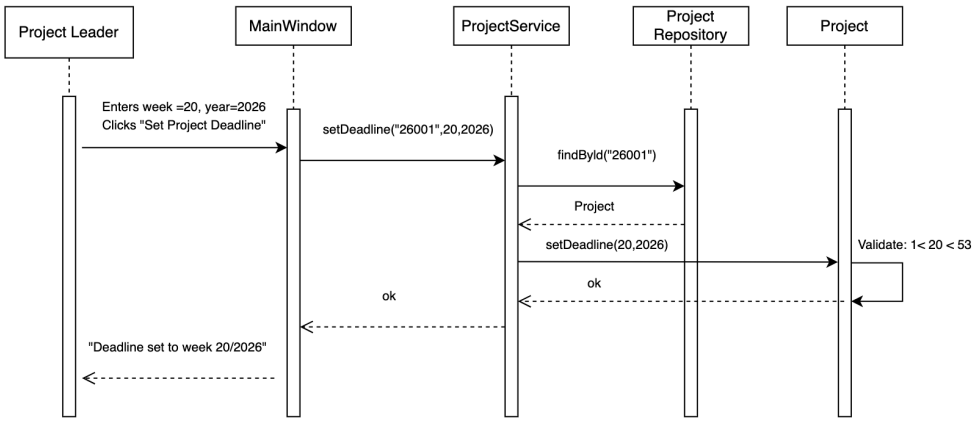
4.9 Use Case 9: Set project Deadline(Viktor)

This use case allows a project leader to set a deadline for a project. The deadline is specified as a week number an a year, following the ISO 8601 standard for a week numbering. The deadline gives the project leader an overview of when the project must be completed.

Main Use case: A project exits in a system. The project leader enters a valid week number, between 1 and 53, and a year. The deadline is successfully saved to the project.

The Diagram 10 shows the sequence of interactions when a project leader succesfully sets a deadline for a project. The project leader enters a valid week number, 20, and year, 2026, in the `MainWindow` and clicks "Set Project Deadline". The `MainWindow` calls `setDeadline()` on the `ProjectService`, which retrives the project from `ProjectRepository` using `FindById()`. The project is then returned to the service, which calls `setDeadline(20, 2026)` on the "Project"object. Since its within the validate range, the "project"saves and confirms the deadline.

Fail scenario: The entered week number is outside the valid range, e.g. 99. The system rejects the input and returns an error "Deadline week must be between 1 and 53")



Figur 10: Sequence diagram for Main use case 9

4.10 Use Case 10: View Project Progress(Viktor)

This use case allows a projet leader to view the overall progress of a project. the progress is calculated as the percentage of budgeted hours that have been registered so far. This gives the project leader an overview of how much work has been completed compared to the total planned effort.

Main use case: A project exits with one or more activities that have budgeted hours. developers have registered time on the activities. the system calculates and displays the progress as a percentage using formula:

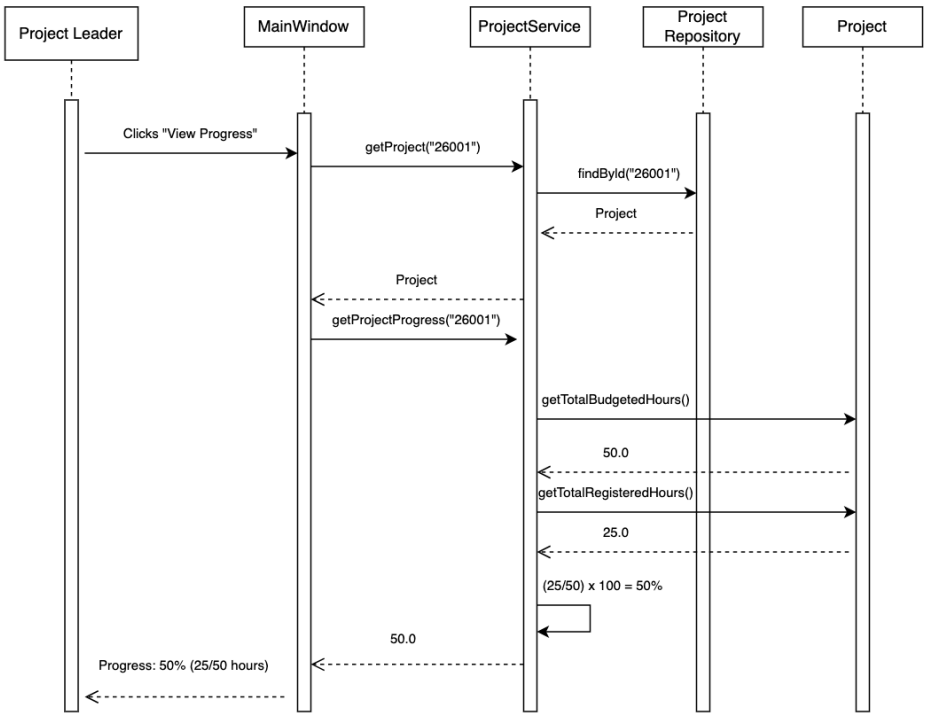
$$\text{Progress} = \frac{\text{Total Registered Hours}}{\text{Total Budgeted Hours}} \times 100\% \tag{1}$$

For example, if 25 out 50 budgeted hours have been registered for progress is 50%.

The Diagram 11 shows the sequence of interactions when a project leader views the progress of a project. The project leader clicks "View Progress"in the `MainWindow`. The `MainWindow` first retrieves the project via `getProject()`, which calls `FindById()` on `ProjectRepository`. The project is returned on the `MainWindow`, which then calls `getProjectProgress()` on `ProjectService`. the service retrieves the total budgeted hours, 50, an toal registered hours, 25, from the "Project"object, and calculates the progress.

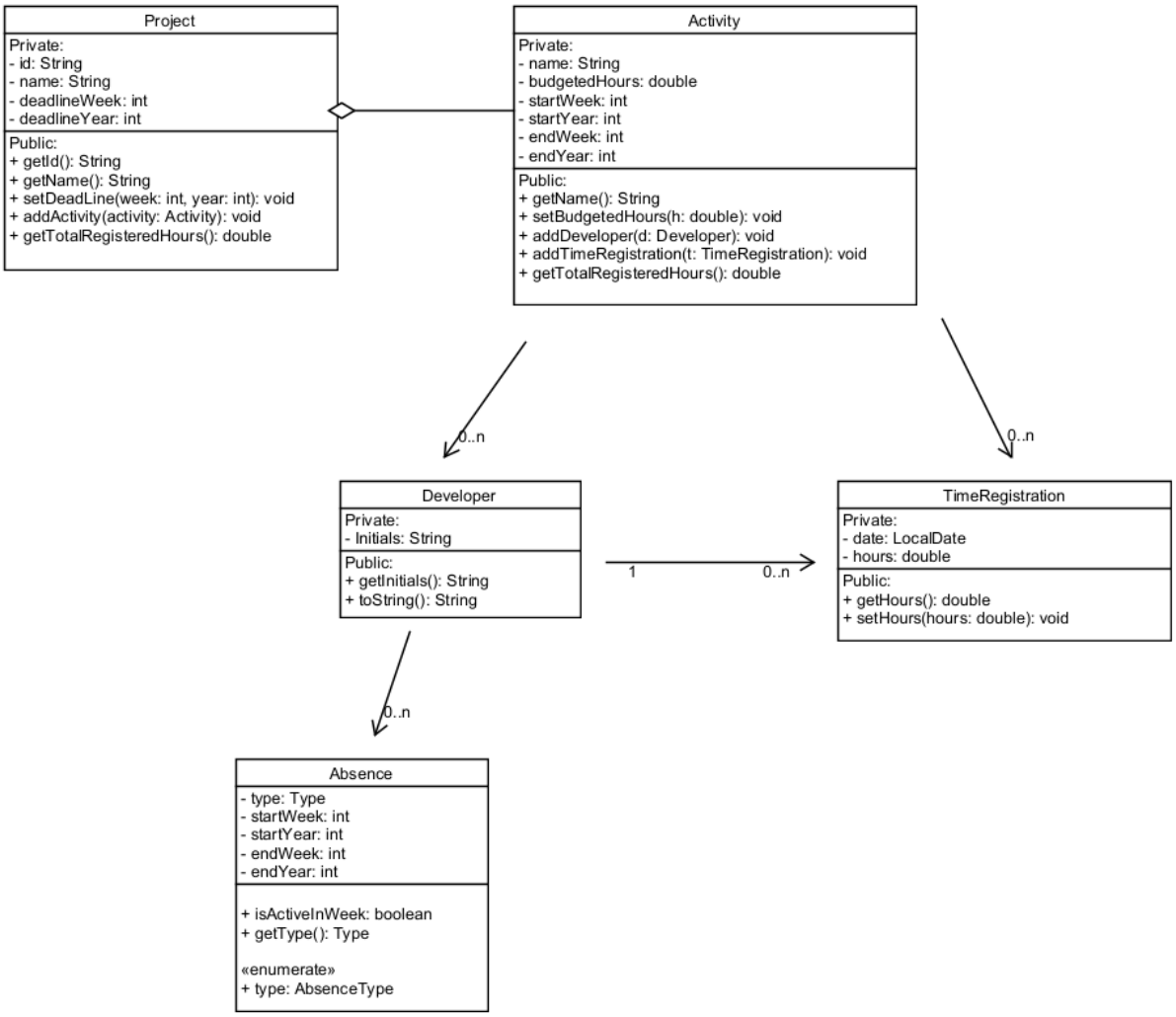
resulting in 50 hours being returned to the `MainWindow`, which displays the progress to the project leader as "Progress: 50%"

Fail scenario: The project has no budgeted hours, e.g. no activities have been created or no budget has been set. The system return a progress of 0% to avoid division by zero.



Figur 11: Sequence diagram for Main use case 10

5 UML Diagram (Nicolai)



Figur 12: UML Diagram

5.1 UML domains (Nicolai)

Illustrated in 12, the UML-diagram shows the central domain model of the program and the relation between the 4 main classes.
The purpose of the UML-digram is to illustrate how projects, activities, developer, time registration and abcence are structured at interact with eachother.

5.1.1 Project

The `Project` class represents the main project entity in the system. When a project is made, attributes such as id, name, and deadlines are initialized and confirmed. In the current

design, the class `Activity` is also modeled as a part of `Project`. This means that `Activity` is created, stored and accessed through `Project`.

5.1.2 Activity

The `Activity` class represents a task or work unit within a project. The class' attributes includes name of developers, budgeted hours and when the activity was started and expected to end.
As mentioned above, this is initiated and managed inside the `Project` class, and an activity is structured inside the main project. In addition, `Activity` is associated with both `Developer` and `TimeRegistration`, since developers are assigned to activities and time registrations are used to record the hours spent on a given activity.

5.1.3 Developer

The `Developer` class represents a registered developer in the system. Each developer puts in their initials and a unique ID is stored. The `Developer` class is associated with `Activity`, `TimeRegistration`, and `Absence`, since developers can be assigned to activities, register working hours, and record periods of absence.

5.1.4 Time Registration

`TimeRegistration` represents time logged by a developer on a specific activity. It includes attributes such as hours, date, and year. This class makes it possible to record and track how much time a developer has spent on a given activity. Furthermore, `TimeRegistration` is associated with both `Developer` and `Activity`, since each registration reflects work performed by a developer on a particular activity.

5.1.5 Absence

`Absence` represents a developer's registered absence in the system. It allows developers to specify the type of absence and the expected duration of their unavailability. The class includes attributes such as week, year, start/end of absence, which are similar to those used in time registration. In addition, `Absence` is associated with `Developer`, as it is used to model periods where a developer is unavailable for project work

6 Program Design (Vedanta)

In addition to the domain classes above, the system also has the Service layer, Repository layer, and the User Interface layer.

The service layer manages the system's use cases and applies the encoded logic. Such as creating projects, assigning developers, and registering work hours.

The repository layer manages the storage and retrieval of critical information about projects, activities, and developers. The system stores this data using a simple data structure rather than an external database.

The user interface layer consists of a simple interface, which is crucial for allowing users to interact with the system. The interface collects input from the user and forwards requests to the service layer, and then displays the results returned by the system.

7 Discussion (Vedanta)

This project takes several assumptions in order to make a manageable system that follows the requirements. Meaning that the end product is a simple planning and registration system, instead of a complex project management system.

Firstly, all employees are assumed to be trusted and known users. This explains the lack of access control or authentication measures taken by the system, such as password protection.

Secondly, the system uses "In Memory storage", meaning that the data is stored in the RAM of the computer. This allows the system to access the stored data much faster and more simply than disk based storage, however, this is a volatile method of storing, since the RAM is flushed everytime the computer is turned off.

Lastly, project progress is tracked using a rudimentary method by applying the ratio of estimated to registered hours. This could be done with more advanced methods such as deadlines, priorities, and dashboards. However, these methods are outside the scope of the project.

8 Appendix

Figurer

1 User-Case diagram over scenarios/fail scenarios 4
2 Sequence diagram for Main use case 1 6
3 Sequence diagram for Main use case 2 7
4 Sequence diagram for Main use case 3 8
5 Sequence diagram for Main use case 4 9
6 Sequence diagram for use case 5 10
7 Sequence diagram for Use Case 6 11
8 Sequence diagram for use case 7 12
9 Sequence diagram for use case 8 13
10 Sequence diagram for Main use case 9 14
11 Sequence diagram for Main use case 10 15
12 UML Diagram 16

Litteratur